

APM Template Design Analysis

Layout patterns for CloudWatch APM visualization in chat-based UI

Project	AWS APM Claude Plugin + Quick App
Scope	MCP-UI template architecture for Cloudscape-rendered widgets
Viewport	~600-800px width, vertical scroll, MCP-UI iframe
Widget Types	stat_card, sparkline, timeline, table, trace_waterfall, log_viewer, change_event_list
Date	April 29, 2026

1. Current Template Assessment

Phase 1 implemented 3 templates: Single, Stacked, and Grid. These were chosen during the architecture debate (Priya debate) as a minimal starting set. This analysis evaluates whether they are correct, what is missing, and what the optimal final set should be.

1.1 Existing Templates

Template	Structure	Slots	Use Case
Single	1 full-width widget	primary(1, any)	Trace waterfall, single chart, single table
Stacked	2-3 vertical widgets	components(max 3, any)	Investigation flow: metric > table > logs
Grid	KPI cards row + 1 primary	cards(max 6, stat_card) + primary(1)	Service overview, health dashboard

1.2 Assessment

All 3 are structurally sound and correctly named for their purpose:

- ✓ Single correctly handles focused deep-dives (traces, single visualizations)
- ✓ Stacked correctly handles multi-step investigations (metric > evidence > root cause)
- ✓ Grid correctly handles overview dashboards (KPI summary + detail table/chart)

Missing patterns identified:

- ✗ Cards-only layout for quick health checks (no primary widget needed)
- ✗ Side-by-side comparison layout (2-column)
- ✗ Full dashboard with cards + multiple stacked widgets (superset of Grid + Stacked)

2. Template Candidate Exploration

12 candidate templates were evaluated. The goal: identify structurally distinct layouts vs. variants that are really just different widget selections in existing slots.

2.1 Full Candidate Catalog

ID	Name	Structure	Classification	Use Case	Verdict
T1	Single	1 full-width widget	Existing	Trace, chart, table	KEEP
T2	Stacked	2-3 vertical widgets	Existing	Investigation flow	KEEP
T3	Grid	Cards row + primary	Existing	Service overview	KEEP
T4	KPI Strip	Cards only, no primary	New, distinct	Quick health check	ADD
T5	Hero	Large chart + context cards	Variant of Grid	Trend analysis	SKIP
T6	Comparison	2 columns side by side	New, distinct	A vs B comparison	ADD
T7	Dashboard	Cards + 2-3 widgets	New, superset	Full dashboard	PHASE 2
T8	Alert Context	Severity banner + evidence	Variant of Stacked	Alarm response	SKIP
T9	Correlation	2 synced time series	Variant of Stacked	Cause analysis	MAYBE
T10	Timeline Focus	Cards + timeline primary	Variant of Grid	Change tracking	SKIP
T11	Log Focus	Cards + log viewer	Variant of Grid	Error investigation	SKIP
T12	Deep Trace	Waterfall + correlated logs	Variant of Stacked	Trace analysis	SKIP

2.2 Key Insight: Templates Define Structure, Not Content

Templates define STRUCTURE (slot shapes and constraints). Widget selection is a separate LLM decision.

T5 Hero = Grid where the LLM puts a chart in 'primary' and stat_cards in 'cards'

T10 Timeline = Grid where the LLM puts timeline in 'primary'

T11 Log Focus = Grid where the LLM puts log_viewer in 'primary'

T12 Deep Trace = Stacked where the LLM puts waterfall + logs in 'components'

These are NOT different templates. The same Grid or Stacked template handles all of them. The intelligence is in the widget catalog prompt, not the template count.

3. Deduplication Analysis

Each candidate was tested against a structural distinctness criterion: does it require a slot configuration that cannot be expressed by any existing template?

Candidate	Structurally Distinct?	Why / Why Not
T4 KPI Strip	YES	Cards only, no primary slot. Grid always requires a primary. This is a genuinely different shape.
T5 Hero	NO	Grid with positions visually swapped. The slot structure is identical - cards + primary. CSS can handle vertical ordering.
T6 Comparison	YES	2-column layout is fundamentally different from any vertical stacking. No existing template has left/right slots.
T7 Dashboard	YES	Cards + multiple primaries. Grid only allows 1 primary. This is the union of Grid + Stacked, a real superset.
T8 Alert Context	NO	Stacked with a severity-colored first item. Styling, not structure. The LLM can style the first widget as an alert banner.
T9 Correlation	MAYBE	Two charts with shared/synced time axes. Structurally it is Stacked, but the synchronized axis behavior requires template-level coordination.
T10 Timeline	NO	Grid with timeline widget in primary slot. Widget selection decision, not template.
T11 Log Focus	NO	Grid with log_viewer in primary slot. Same structure as Grid.
T12 Deep Trace	NO	Stacked with waterfall + log_viewer. Widget selection, not structure.

4. Recommended Template Set

Based on the structural analysis, the recommended set is 6 templates (hard cap). This covers 95%+ of APM query patterns while keeping the LLM's selection task simple. 6 is the ceiling - no template should be added unless it introduces a genuinely new slot structure.

4.1 Final 6 Templates

1. FOCUS (was Single)

Slots: primary(1, any)

One full-width widget. Maximum space for a single visualization.

Example queries:

- "Show me trace abc123"
- "Error table for auth-service"
- "Lambda invocation chart"

When to use: When the user asks about ONE specific thing and needs maximum detail.

2. INVESTIGATE (was Stacked)

Slots: components(max 3, any)

2-3 widgets stacked vertically. Top-down investigation flow: symptom > evidence > cause.

Example queries:

- "Why is latency spiking?"
- "Debug order processing failures"
- "What is causing 5xx errors?"

When to use: When the answer requires connecting multiple data types to tell a story.

3. OVERVIEW (was Grid)

Slots: cards(max 6, stat_card) + primary(1, any)

KPI cards strip on top, one primary widget below. At-a-glance health + detail.

Example queries:

- "How is payment-svc doing?"
- "Show me the error trend"
- "What changed last hour?"

When to use: When the user wants a service summary: key numbers + one deeper view.

4. STATUS (NEW)

Slots: cards(max 6, stat_card)

Cards only, no primary widget. Compact multi-service health check.

Example queries:

- "Is everything healthy?"
- "Service status across all APIs"
- "Quick check before deploy"

When to use: When the answer is YES/NO per service. No charts or tables needed.

5. COMPARE (NEW)

Slots: left(max 2, any) + right(max 2, any)

Two columns side by side. Each column can hold 1-2 widgets.

Example queries:

- "Compare prod vs staging latency"
- "Service A vs Service B"
- "Before and after deployment"

When to use: When the user explicitly compares two things. Only template with horizontal split.

6. DASHBOARD (NEW)

Slots: cards(max 6, stat_card) + components(max 2, any)

Superset of Overview: KPI cards row + 2 stacked widgets below. Full operational dashboard.

Example queries:

- "Full service dashboard for auth-svc"
- "Give me the complete picture"
- "Incident overview with metrics, logs, and timeline"

When to use: When Overview's single primary is not enough. Cards + chart + table, or cards + timeline + logs.

5. Query Coverage Matrix

Mapping common APM query patterns to templates and widget selections. Every pattern maps to exactly one template. No gaps.

User Query	Template	Widgets Selected by LLM
"Is everything healthy?"	STATUS	3-6 stat_cards (one per service)
"How is payment-svc doing?"	OVERVIEW	stat_cards (latency, errors, throughput) + table (top errors)
"Why is latency spiking?"	INVESTIGATE	sparkline (latency trend) + table (slow endpoints) + log_viewer (errors)
"Show me trace abc123"	FOCUS	trace_waterfall
"Compare prod vs staging"	COMPARE	Left: stat_cards(prod) + sparkline. Right: stat_cards(staging) + sparkline
"Show me the error trend"	OVERVIEW	stat_cards (error count, rate) + sparkline (error trend)
"What changed last hour?"	OVERVIEW	stat_cards (deploy count, config changes) + change_event_list
"Show me recent errors"	OVERVIEW	stat_cards (error count, top error type) + log_viewer (filtered errors)
"Is CPU causing latency?"	INVESTIGATE	sparkline (CPU %) + sparkline (latency) + stat_cards (correlation)
"Alarm: Lambda errors > 5%"	INVESTIGATE	stat_card (alarm status) + sparkline (error rate) + log_viewer
"Analyze distributed trace"	INVESTIGATE	trace_waterfall + log_viewer (correlated logs) + stat_cards
"Quick check before deploy"	STATUS	stat_cards (all services green/red)
"Full dashboard for auth-svc"	DASHBOARD	stat_cards + sparkline (latency) + table (top errors)
"Incident overview with context"	DASHBOARD	stat_cards (severity, duration) + timeline + log_viewer

6. Design Principles Applied

Chat viewport constraint

All templates render in ~600-800px width with vertical scroll. Horizontal multi-column only works for small items (stat_cards in Compare). Tables, charts, logs, and traces always need full width.

Cloudscape alignment

Cloudscape's dashboard pattern uses a 12-column grid with Small (3 cols), Medium (6 cols), Large (9 cols), and XL (12 cols) item sizes. In chat context, all primary widgets are effectively XL. Cards map to Small. The 5 templates respect this by only allowing horizontal layout for cards.

Minimize visual noise (Cloudscape guideline)

One item should serve one definitive user goal. Don't include so much data that it's not easily scannable. This is why we cap at 3 stacked widgets and 6 cards. Cloudscape also recommends max 2 visualization types per item.

Template vs. widget selection separation

Templates define slot shapes and constraints. Widget selection is a separate LLM decision based on the widget catalog prompt. This separation keeps both concerns simple and independently evolvable.

Academic foundation (Dashboard Design Patterns, 2022)

Research identifies 3 layout archetypes: Stratified (49% of dashboards, top-down), Table (19%, grid alignment), and Grouped (33%, semantic clustering). Our 5 templates map to: Focus/Investigate/Overview = Stratified, Status = Table, Compare = Grouped.

Eval-driven expansion

Start with 5. Run evals on 50-100 sample queries. If >15% of queries need a structure none of the 5 can express, add the Dashboard template (Phase 2 candidate). Never add a template just because a widget combination looks different. Only add when the slot structure is genuinely new.

7. Implementation Plan

Step 1: Rename existing templates

Single > Focus, Stacked > Investigate, Grid > Overview. Update JSON files and engine references. Names should reflect purpose, not layout mechanics.

Step 2: Add Status template

New JSON template: cards(max 6, stat_card), no primary slot. Simplest template - just renders a responsive grid of stat cards.

Step 3: Add Compare template

New JSON template: left(max 2, any) + right(max 2, any). CSS grid with 2 equal columns. Container queries handle narrow viewports by stacking.

Step 4: Migrate remaining 5 widget types

sparkline, timeline, trace_waterfall, log_viewer, change_event_list. Wrap each in Cloudscape Container component with your design tokens.

Step 5: Write widget catalog prompt

Structured document describing each widget type: what it shows, when to use it, data requirements, constraints. Include template selection rules. This is the core IP of the product.

Step 6: Run evals

50-100 sample APM queries. Check template selection accuracy and widget selection quality. Iterate on catalog prompt. Decide whether to add Dashboard template based on gap data.